ENGINEERING WHITEPAPER

By Dron Pancholi

# The Architecture AI Cannot See

*Why AI coding agents read your software but do not understand it - and what a correct solution requires*

> *"Every production system has two architectures:*
> *the one in the files, and the one that lives nowhere.*
> *We taught AI to read the first.*
> *The second is the one that matters."*


| | |
|---|---|
| AUTHOR | **Dron Pancholi · dronpancholi@gmail.com** |
| DOMAIN | Software Engineering · AI Systems · Program Comprehension |
| AUDIENCE | Principal Engineers · Architects · Engineering Leaders |
| META | 12 - 15 min read · Version 1.0 · July 2026 |



Independent engineering research paper by Dron Pancholi.
Establishes a problem framing and design requirements.

Public · 2026

BEFORE YOU READ

This document introduces a mental model - one that changes how we evaluate AI coding tools, and why the current generation of file-based agents operates on an incomplete software substrate.

TABLE OF CONTENTS

# Contents


I **The Capability We Celebrate Is Not the Capability We Need** . . .
*Code generation vs. safe modification*

II **Why More Context Doesn't Close This Gap** . . .
*The context ceiling and limits of scale*

III **The File Is Not the Software** . . .
*The Representation Gap*

IV **The Architecture That Lives Nowhere** . . .
*The Second Architecture & Scenario A*

V **Semantic Drift: How AI Makes Codebases Incoherent** . . .
*Stateless sessions & Scenario B*

VI **What a Correct Solution Must Provide** . . .
*The Knowledge Surface & 6 properties*

VII **A Conceptual Architecture** . . .
*The Repo-IR 3-layer reasoning model*

VIII **What Makes This Hard & Counterarguments** . . .
*Intellectual bounds & primary objections*

IX **The Questions That Matter** . . .
*Sharpening core research questions*

X **If This Thesis Is Correct...** . . .
*Six falsifiable engineering predictions*

*Manifesto Posters · The Repository Stack Framework · References · Closing Statement*

# *"The file is not the software."*

*The software is the knowledge the file was written to hold.*

**DRON PANCHOLI · ENGINEERING MANIFESTO**

I · ENGINEERING NOTE

# The Capability We Celebrate Is Not the Capability We Need

*The question is not whether AI can write code. The question is whether it can understand code it didn't write.*

AI coding agents write functions, generate unit tests, and scaffold modules faster than any human team. Benchmark scores rise quarterly. Demos are compelling.

Yet every senior engineer deploying these tools into production notices the same pattern within days: the agent fixes one function and silently breaks two others. It adds a feature that is locally correct but architecturally incompatible. It passes all tests while violating an invariant no test checks.

This is not a performance flaw, a context length limitation, or a model size issue. It is a fundamental representation problem.

*AI coding agents fail not because they are unintelligent, but because they are reading the wrong representation.*

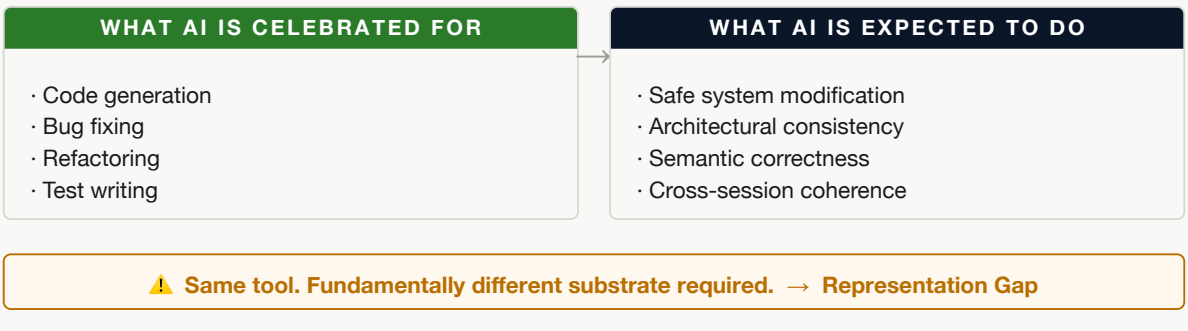



*Figure 1.1 - The Capability Gap: Code generation and architectural reasoning require fundamentally different substrates.*

The gap between what we celebrate - code generation - and what we need - safe modification - is an architectural divide. Generating code requires syntax. Modifying software safely requires system-wide comprehension.

> **TAKEAWAY**
>
> The bottleneck in modern software engineering is no longer writing code. It is understanding code well enough to modify it safely.

II · ENGINEERING NOTE

# Why More Context Doesn't Close This Gap

*A larger window onto the wrong representation is still the wrong representation.*

When agents fail architecturally, the intuitive reaction is simple: feed them more context. Add more modules, include the test suite, expand the context window to millions of tokens.
This reaction is understandable. It is also wrong.






*Figure 2.1 - The Context Processing Pipeline: Sequential token windows strip away network structure.*

Context windows, regardless of scale, process a flat sequence of tokens. But software is not a sequence. It is a network - a network of typed dependencies, behavioral contracts, and temporal decisions.

You cannot flatten a network into a sequence without destroying the relationships that define it. Human experts do not read files sequentially; they construct a mental graph. Sequence readers cannot produce graphs.

| Approach | What it provides | What it misses |
|---|---|---|
| Small Context Window | Local file neighborhood | System-wide context |
| Large Context Window | Raw repository text | System structure & invariants |
| Knowledge Surface | Graph-structured intelligence | Nothing - this is the goal |

*Figure 2.2 - Scale vs. Structure: Scale expands token quantity; structure provides comprehension.*

> **TAKEAWAY**
>
> Context is not comprehension. Comprehension requires structure, not scale.

III · ENGINEERING NOTE

# The File Is Not the Software

*Files were designed for humans. We are feeding them to machines and calling the result understanding.*

The file system was invented as a human retrieval interface. Files have names so people can navigate directories. They were never intended to be a machine-native representation of software.

Yet every modern AI coding agent treats files as the primary unit of software intelligence.

| Software contains… | In files? |
|---|---|
| Syntax and structure | Stored |
| Import relationships | Stored |
| Behavioral invariants | Not stored |
| Capability contracts | Not stored |
| Layer rules and conventions | Not stored |
| Design decision history | Not stored |
| Rejected alternatives | Not stored |

*Files contain the fraction of software that was easy to write down.*

*Figure 3.1 - Storage Dichotomy: Files store syntax; software consists of unwritten behavior and contracts.*

**CANONICAL CONCEPT** **The Representation Gap**

The structural distance between what is stored in source files and what is required to safely modify the system.

*Files contain the fraction of software that was easy to write down.*

**TAKEAWAY**

The file is not the software. It is a human-readable projection of the software onto a storage medium.

# *"Context is not comprehension."*

*A larger window onto a flat sequence is still a sequence.*

DRON PANCHOLI · ENGINEERING MANIFESTO

ICONIC FRAMEWORK · THE REPOSITORY STACK

# The Representation Hierarchy

*Where current AI tools stop, and where real software understanding actually lives.*

**1. REAL SOFTWARE**

System Behavior · Domain Invariants · Capability Contracts · Design Intent

*compiled into*

↓

**2. KNOWLEDGE SURFACE (The Repository IR)**

Structured Property Graph · Queryable Invariant Registry · Architectural Memory

⚠ THE FILE BOUNDARY · CURRENT AI TOOLS OPERATE ENTIRELY BELOW THIS LINE

**3. FILES, TOKENS & CONTEXT WINDOWS**

Source Code Text Files · Flat Token Sequences · Pattern Matching

**TAKEAWAY**

Current AI tools treat Level 3 (Files) as the whole problem. Real engineering comprehension requires bridging Levels 1 and 2.

IV · ENGINEERING NOTE

# The Architecture That Lives Nowhere

*The knowledge that makes software understandable is the knowledge that isn't written down.*

Every production system has two architectures.

**The First Architecture** lives in files: classes, functions, and import statements. It can be parsed and indexed. It is what static linters read and what AI agents receive.

**The Second Architecture** lives nowhere. It is the unwritten network of invariants, capability contracts, and historical design choices. It is what senior engineers know and what junior engineers spend months learning.

ENGINEERING SCENARIO

**Scenario A · Permission Model Divergence**

An engineer asks an AI agent to update role-based authorization in an API controller. The agent updates the controller methods perfectly, passing unit tests. However, an unwritten invariant dictates that authorization checks must also mirror in async background worker queues.

*Result: The API controller is secure, but the background worker processes unvetted payload queues in production - a silent privilege escalation zero tests caught.*

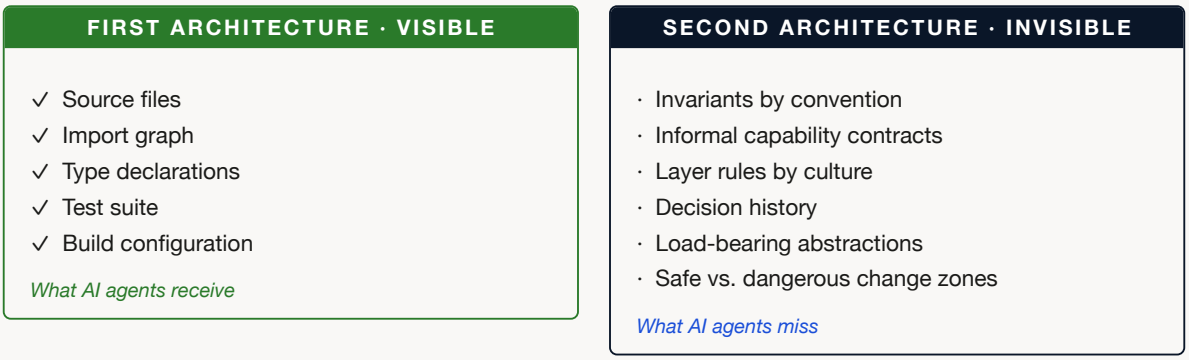



*Figure 4.1 - Dual Architectures: Static tools parse the First Architecture; safe engineering depends on the Second.*

When a senior developer departs, the Second Architecture is what leaves with them. File-based AI agents operating exclusively on the First Architecture produce changes that are locally syntactically valid yet globally architecturally destructive.

**CANONICAL CONCEPT** **The Second Architecture**

The unwritten network of invariants, layer boundaries, and historical decisions that determines whether a system modification is safe.

**TAKEAWAY**

The most valuable knowledge in a software system is the knowledge that lives in no file.

V · ENGINEERING NOTE

# Semantic Drift: How AI Makes Codebases Incoherent

*The agent makes every change correctly. The codebase becomes wrong.*

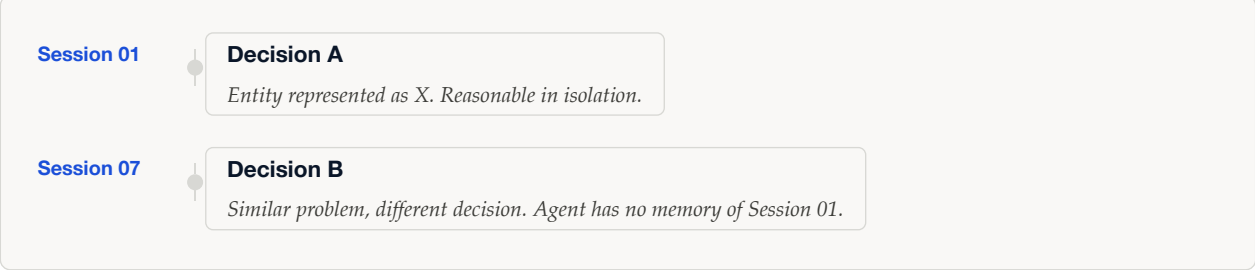
There is a silent failure mode in AI-assisted development: **semantic drift**. Stateless agents make locally reasonable decisions in isolation that gradually undermine global coherence.

**ENGINEERING SCENARIO**

**Scenario B · Async Schema Contract Mismatch**

An agent refactors an internal user entity schema in session 4, renaming `billing_address` to `payment_location`. The change compiles and passes local tests. In session 12, asked to consume user events in an analytics worker, the agent reads the old entity docs and writes logic expecting `billing_address`.

*Result: Both sessions pass tests in isolation, but production event processing drops analytics events silently for weeks.*



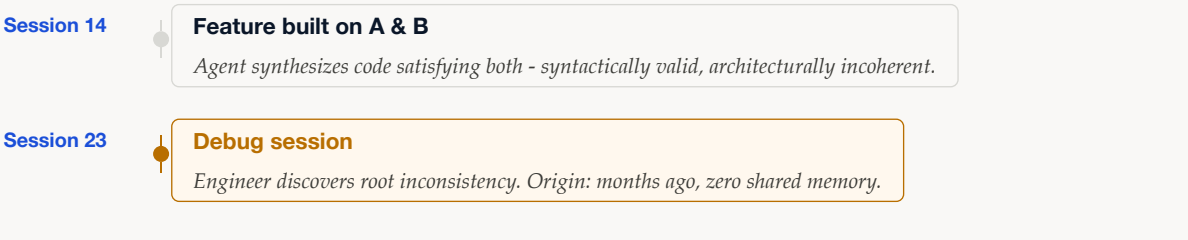



*Figure 5.1 - Semantic Drift Timeline: Stateless sessions accumulate architectural incoherence over time.*

*Wrong code fails fast; semantically drifted code passes tests and breaks production.*

**TAKEAWAY**

Without architectural memory, every AI session is session one. The codebase pays the price.

VI · ENGINEERING NOTE

# What a Correct Solution Must Provide

*The goal is not a better agent. The goal is a richer substrate for the agent to reason over.*

Solving the representation gap requires a fundamental substrate shift - a machine-legible representation capable of explicitly capturing both architectures.

**Six Required Properties of a Correct Software Representation**

① **System-scope**
Reason across files, not within files. Entities, not paths.

② **Semantic typing**

Distinguish structural, architectural, and capability dependencies.

③ **Invariant tracking**
Maintain the Second Architecture as a living, queryable artifact.

④ **Temporal persistence**
Architectural memory survives across sessions. Drift is detectable.

⑤ **Traceability**
Every entity answers: why does this exist? What breaks without it?

⑥ **Honest uncertainty**
Known, inferred, and unknown are explicit. Never false confidence.

*Figure 6.1 - The Six Essential Properties of a Software Knowledge Substrate.*

CANONICAL CONCEPT **The Knowledge Surface**

A structured, queryable Repo-IR representing both written syntax and unwritten invariants of a software repository.

*Build the representation layer, and the reasoning follows.*

**TAKEAWAY**

The correct solution is not a smarter LLM. It is a richer software representation substrate.

VII · ENGINEERING NOTE

# A Conceptual Architecture

*Software repositories require an intermediate representation the same way compilers do.*

Compilers do not translate C directly to machine code; they compile source into an Intermediate Representation (IR) to optimize and analyze logic abstractly. Software reasoning tools require an equivalent Repo-IR.

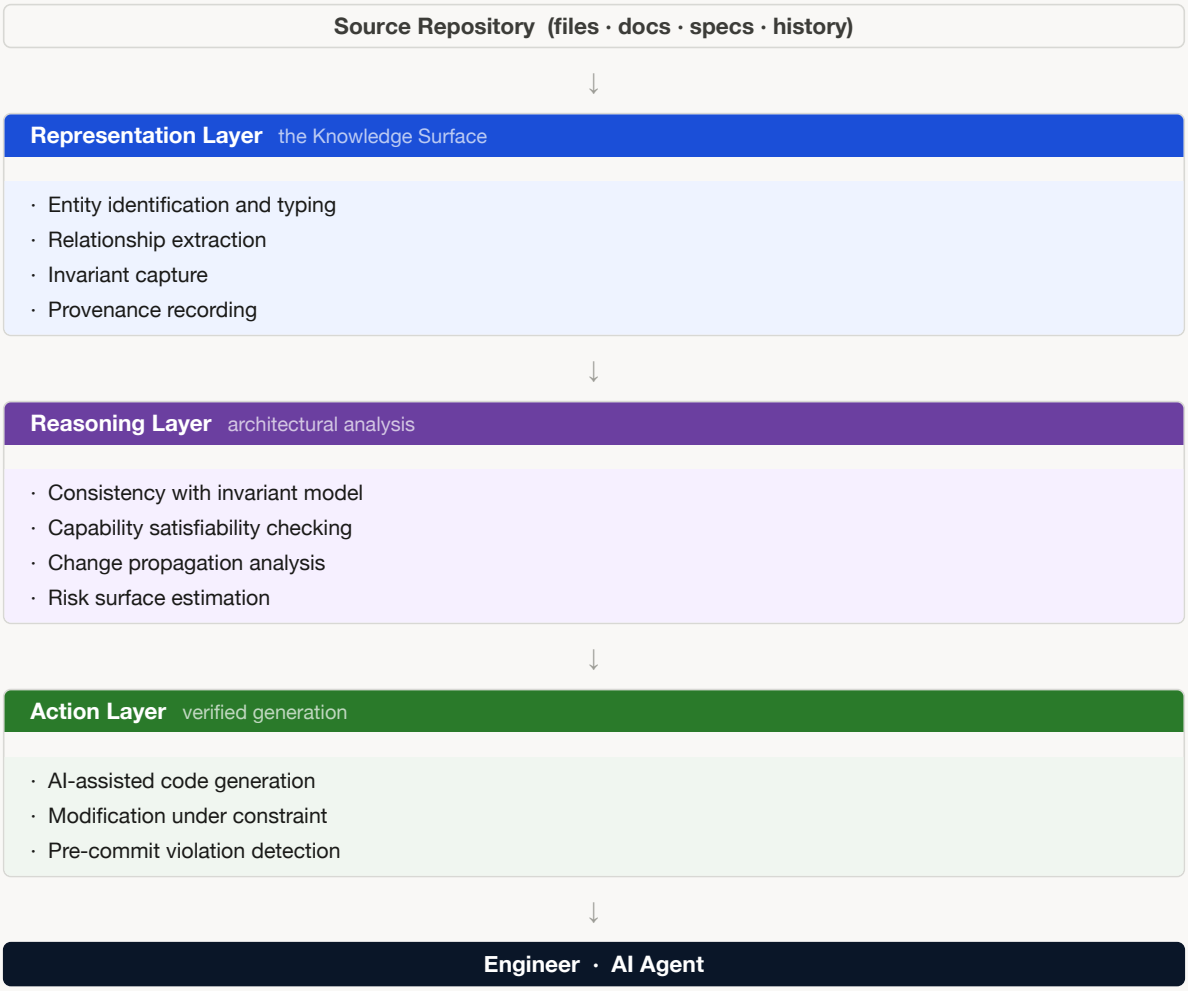



*Figure 7.1 - The Three-Layer Architecture: Decoupling representation, reasoning, and code generation.*

By inserting a Representation Layer (Knowledge Surface) between source files and AI reasoning, proposed modifications are evaluated against invariant models before code is generated or committed.

*In current workflows, architectural violations are discovered after commits in production. In a 3-layer architecture, violations are caught before code generation occurs.*

**TAKEAWAY**

Give AI a better substrate, not better prompts. Substrate determines capability.

VIII · ENGINEERING NOTE

# What Makes This Hard

*The primary challenge is not technical execution. It is epistemological honesty.*

Constructing a software Knowledge Surface presents hard trade-offs. Higher architectural fidelity requires higher construction and maintenance costs.

| Representation Dimension | Construction Cost | Maintenance Cost | Value Certainty |
|---|---|---|---|
| Syntactic structure | Low | Low | Well understood |
| Import dependencies | Low | Low | Well understood |
| Semantic relationships | Medium | Medium | Context-dependent |
| Capability contracts | High | Medium | Significant |
| Architectural invariants | Very High | High | Significant |
| Decision provenance | Medium | Low | High - if captured |

*Figure 8.1 - Architectural Trade-Offs Matrix: Higher value dimensions carry higher extraction costs.*

Four fundamental engineering hurdles exist: cold-starting legacy codebases, belief revision during continuous code updates, extracting tacit human team knowledge, and calibrating model confidence.

> **TAKEAWAY**
>
> A model that reports uncertainty is safe. A model with false architectural confidence is dangerous.

VIII-B · ENGINEERING NOTE

# Intellectual Honesty & Counterarguments

*A thesis gains authority not by claiming absolute truth, but by explicitly bounding its scope.*

## What This Paper Is NOT Claiming

**× Larger context windows are useless.**

Context window expansion is a genuine technical achievement that enables broader single-file retrieval and multi-file text inspection. This paper asserts only that context length cannot alter the sequential nature of tokens into a graph representation.

**× LLMs are incapable of reasoning.**

Large language models display remarkable capabilities in local logical synthesis and structural transformation. The architectural bottleneck is not neural intelligence, but the representation substrate fed into the model.

**× Current AI coding tools provide no engineering value.**

Modern AI coding assistants save software teams millions of developer hours on boilerplate generation, documentation, and local scaffolding. The boundary of failure occurs during complex, unconstrained cross-file refactoring.

**× Static analysis tools should be replaced by neural models.**

Static analysis compilers remain essential. The thesis argues for the unification of static analysis graphs and neural inference into a single queryable Repository Intermediate Representation (Repo-IR).

## The Strongest Counterargument

**PRIMARY OBJECTION**

***"Won't fine-tuning LLMs on ASTs, commit histories, and execution traces allow them to implicitly learn the Second Architecture without needing an explicit Repo-IR?"***

This is the strongest opposing argument. However, execution traces and ASTs record past executions; they do not record which choices were rejected, which invariants are load-bearing, or which layer rules are strict conventions versus historical anomalies. Just as compilers could not eliminate intermediate representations by training on raw binary execution dumps, neural agents cannot bypass explicit representation substrates without incurring exponential architectural drift.

*Representation sets the ceiling of reasoning. No model can reason about structural relationships that do not exist in its input substrate.*

**TAKEAWAY**

Intellectual honesty requires acknowledging that AI tools excel at syntax while requiring a dedicated representation layer for architecture.

IX · ENGINEERING NOTE

# The Questions That Matter

*We do not close with answers. We close by sharpening the open research questions.*

The future of software AI tools depends on how rigorously our field addresses four core research questions.

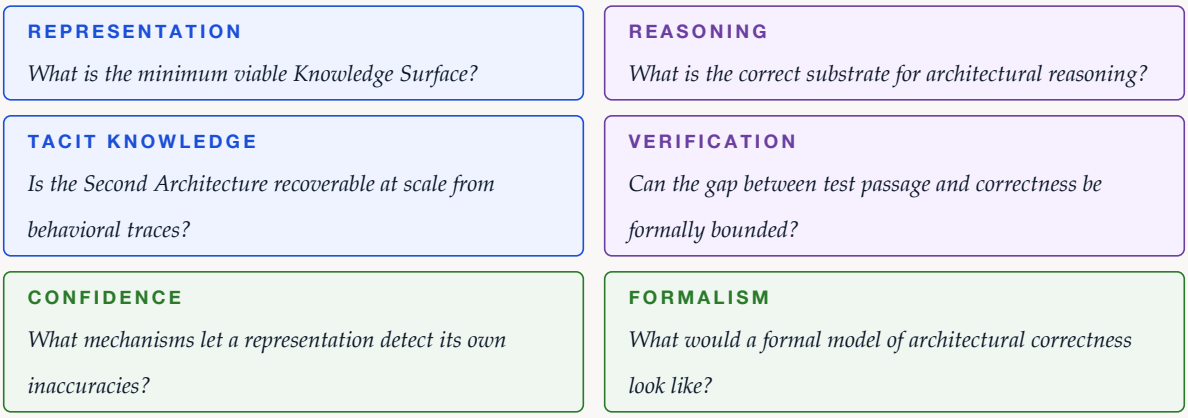



*Figure 9.1 - Open Research Questions by Domain.*

RESEARCH QUESTION 1

**Software Intermediate Representation (Repo-IR)**

Can we formally define an IR that decouples source files from architectural reasoning target rules?

RESEARCH QUESTION 2

**Safe Agent Refusal Policy**

What exact threshold determines when an AI agent must refuse a requested change due to invariant conflict?

RESEARCH QUESTION 3

**Tacit Knowledge Recovery**

Can unwritten team conventions be reliably extracted from git commit logs, code reviews, and historical telemetry?

RESEARCH QUESTION 4

**Formal Architectural Semantics**

What does a formal semantics of architectural correctness look like beyond standard type systems and linters?

**TAKEAWAY**

Answering these four questions will define the next decade of software engineering tooling.

X · ENGINEERING NOTE

# If This Thesis Is Correct...

*Six falsifiable engineering predictions for the next decade of AI software tooling.*

If the core thesis of this paper holds - that AI coding tools operate on an incomplete representation of software - several observable shifts must occur over the coming decade.

These are not marketing claims. They are falsifiable engineering predictions with clear falsification criteria.

**FALSIFIABLE PREDICTION 1** — **The Context Window Plateau**

**Context windows will expand to millions of tokens, yet agentic architectural failure rates in production codebases will remain essentially unchanged.**

| | |
|---|---|
| **WHY** | Expanding sequence capacity does not transform a flat token stream into a typed graph representation. |
| **SIGNAL** | Enterprise surveys reporting persistent architectural regression rates despite using 2M+ token window models. |
| **FALSIFICATION** | *A file-based model achieving a 90%+ reduction in cross-file invariant violation rates solely through context window expansion.* |

**FALSIFIABLE PREDICTION 2** — **From Retrieval to Representation**

**Developer tools will transition from retrieval-augmented generation (RAG over raw file chunks) to representation-driven reasoning (Repo-IR).**

| | |
|---|---|
| **WHY** | Retrieval returns file text; modification requires typed dependency and invariant graphs. |
| **SIGNAL** | Major developer tool vendors shipping persistent repository knowledge graphs rather than vector databases. |
| **FALSIFICATION** | *Vector-RAG file retrieval remaining the dominant state-of-the-art architecture for autonomous coding agents in 2030.* |

**FALSIFIABLE PREDICTION 3** — **Convergence of Compilers and Neural Models**

**Static analysis compilers and LLMs will merge into unified repository intermediate representations (Repo-IR).**

| | |
|---|---|
| **WHY** | Neural models require structural constraints to prevent drift, while static analysis requires neural inference to infer unwritten rules. |
| **SIGNAL** | New open-source Repo-IR standards adopted by major IDEs and compiler toolchains. |
| **FALSIFICATION** | *Static analysis and neural LLMs remaining completely isolated tools with zero shared intermediate representation.* |

**FALSIFIABLE PREDICTION 4** — **Repository Intelligence as a Discipline**

**Engineering teams will explicitly maintain 'Knowledge Surfaces' with the same operational rigor currently applied to CI/CD and type systems.**

| | |
|---|---|
| **WHY** | Unwritten invariants must be formalized into queryable artifacts to survive developer churn and agentic modification. |
| **SIGNAL** | The emergence of 'Repository Knowledge Engineer' roles and automated invariant extraction CI gates. |
| **FALSIFICATION** | *Organizations abandoning internal architectural documentation and relying 100% on unconstrained prompt generation.* |

**FALSIFIABLE PREDICTION 5** — **Substrates Will Outvalue Prompts**

**The competitive moat for AI engineering tools will reside in their repository representation substrate, not model size or prompt engineering.**

| | |
|---|---|
| **WHY** | Model intelligence is commodity infrastructure; software knowledge representation is domain context. |
| **SIGNAL** | Developer tool valuations driven by repository indexing and representation accuracy rather than underlying LLM wrappers. |
| **FALSIFICATION** | *Generic raw-prompt wrappers outperforming representation-backed agent systems on complex multi-file benchmarks.* |

**FALSIFIABLE PREDICTION 6** — **The Semantic Drift Crisis**

**Codebases developed primarily by unconstrained file-based AI agents will suffer severe architectural incoherence within 24 months of deployment.**

| | |
|---|---|
| **WHY** | Stateless agents make locally correct decisions that continuously contradict prior unwritten conventions. |
| **SIGNAL** | A sharp rise in refactoring contracts and architectural debt remediations for agent-scaffolded projects. |
| **FALSIFICATION** | *Agent-generated production codebases maintaining equal or better long-term architectural coherence without representation gates.* |

*A prediction is only valuable if reality can prove it wrong.*

**TAKEAWAY**

> Track these predictions by observing production codebase health as AI code generation accelerates.

RESEARCH POSITIONING · LITERATURE FOUNDATIONS

# Theoretical Foundations & Further Reading

This whitepaper positions software AI tools within established traditions of computer science research. Readers interested in exploring the theoretical foundations of software representation are directed to six key literature areas:

PROGRAM COMPREHENSION

### Essential Complexity & Mental Models

*Brooks, F. P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. IEEE Computer.*

Establishes that the hard part of building software is the conceptual craft of design, not accidental representation in syntax.

COMPILER DESIGN

### Intermediate Representations for Decoupled Analysis

*Lattner, C., & Adve, V. (2004). LLVM: A Framework for Lifelong Program Analysis & Transformation. CGO.*

Demonstrates how decoupled IR substrates enable program analysis and transformations impossible on raw source text.

SOFTWARE ARCHITECTURE

### Information Hiding & Modular Invariants

*Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. CACM.*

Formalizes that modular boundaries exist to hide design decisions - decisions that files alone fail to capture for machines.

REPOSITORY MINING

### Architecture Recovery & Tacit Knowledge

*Canfora, G., & Di Penta, M. (2009). New Frontiers of Reverse Engineering. IEEE TSE / MSR.*

Pioneers the recovery of hidden architectural conventions and tacit team decisions from developer commit histories.

COGNITIVE PSYCHOLOGY

### Empirical Programming Knowledge & Schemas

*Soloway, E., & Ehrlich, K. (1984). Empirical Studies of Programming Knowledge. IEEE TSE.*

Shows that expert programmers utilize high-level tacit schemas and invariants rather than line-by-line syntax readers.

GRAPH PROGRAM SEMANTICS

### Neural Program Representations

*Allamanis, M., Brockschmidt, M., & Khademi, M. (2018). Learning to Represent Programs with Graphs. ICLR.*

Demonstrates that graph-structured representations outperform flat token sequences on semantic code reasoning tasks.

## CLOSING STATEMENT

*There is a version of the future where AI dramatically accelerates software development without making systems less comprehensible. Where tools assist not just with code production, but with architectural coherence.*

*We are not in that future yet.*

*What is missing is not model capacity. What is missing is the conviction that the representation layer deserves the same quality of engineering attention that has been given to the generation layer.*

***The file is not the software.***
***The software is the knowledge the file was written to hold.***
***Until we build systems that understand that distinction,***
***we are handing AI a shadow and calling it the thing.***

**Dron Pancholi · dronpancholi@gmail.com**